

HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

Object-Oriented Programming Assignment Report

Table of Contents

1. Problem Statement
2. Objective
3. Requirements and Environment Used
4. Design / Proposed Solution
5. Algorithm / Pseudocode / Flowchart
6. Implementation / Source Code
7. Test Cases and Expected / Actual Results
8. Execution Screenshots / Output
9. Analysis and Discussion
10. Conclusion
11. Individual Contribution of Group Members
12. References
13. One-Page Write-Up

1. Problem Statement

The hotel industry handles a large amount of information related to guests, rooms, reservations, check-in, check-out, and billing. Managing these activities manually using registers or spreadsheets can be time-consuming and may lead to errors such as incorrect guest details, double booking of rooms, difficulty in tracking room availability, and inaccurate billing. As the number of guests and rooms increases, maintaining and retrieving reservation information becomes more difficult. Therefore, an efficient computerized system is required to simplify and automate hotel accommodation and reservation activities. The Hotel Accommodation and Reservation Management System using C++ is proposed to provide an organized solution for managing hotel operations. The system allows hotel staff to maintain guest information, manage room details, check room availability, create and cancel reservations, allocate rooms, and record check-in and check-out details. It can also calculate the total accommodation cost based on room type, number of days, and applicable charges, thereby reducing manual calculation errors. The system is developed using C++ and Object-Oriented Programming (OOP) concepts. Classes and objects can be used to represent guests, rooms, reservations, and billing information. Encapsulation helps protect important data, while functions are used to perform operations such as booking, cancellation, room allocation, and bill generation. File handling can also be incorporated to store and retrieve reservation and guest records for future use. The primary goal of the proposed system is to improve the efficiency, accuracy, and reliability of hotel reservation management. It helps staff quickly identify available and occupied rooms, maintain accurate guest records, avoid duplicate reservations, and provide faster services to customers. The system also reduces paperwork and saves time by automating repetitive hotel management tasks. The proposed solution can be useful for small and medium-sized hotels, lodges, and accommodation centers. It provides a simple and cost-effective approach to hotel management while demonstrating the practical application of C++ programming and OOP concepts. Overall, the system aims to provide a structured, secure, and user-friendly platform for managing hotel accommodation and reservations efficiently.

2. Objective

The main objective of the Hotel Accommodation and Reservation Management System using C++ is to develop an efficient computerized system for managing hotel rooms, guests, reservations, check-in, check-out, and billing. The system aims to reduce manual effort, improve accuracy, and provide quick access to hotel-related information. It also demonstrates the practical implementation of C++ Object-Oriented Programming concepts in a real-world hotel management application.

Key Objectives

1. **Efficient Guest Information Management:**
To maintain complete and organized guest records, including personal details, contact information, room allocation, booking dates, and duration of stay
2. **Effective Room Availability Management:**
To monitor and manage the status of all hotel rooms by identifying available, reserved, occupied, and vacant rooms, facilities.
3. **Automated Reservation and Cancellation:**
To provide an efficient reservation system that allows users to book rooms, modify reservation details
4. **Simplified Check-In and Check-Out Process:**
To maintain accurate records of guest arrival and departure and make the check-in and check-out process faster, more organized
5. **Accurate Billing and Payment Calculation:**
To automatically calculate the total accommodation charges based on room type, number of days stayed, and other applicable charges, reducing calculation errors and improving billing accuracy.
6. **Reduction of Manual Work and Errors:**
To minimize the dependency on paper-based records and manual calculations, thereby reducing data-entry mistakes, duplicate records, and loss of important reservation information.
7. **file handling for developing a structured and maintainable hotel management application.**

3. Requirements and Environment Used

Category	Details
Programming Language	C++17
Programming Concepts	Classes, objects, encapsulation, abstraction, inheritance, virtual functions, runtime polymorphism, smart pointers, STL containers
Compiler	g++ / GCC with C++17 support
IDE / Editor	Visual Studio Code, Code::Blocks, Dev-C++, or any C++17-compatible IDE
Libraries	iostream, vector, memory, iomanip, sstream, algorithm, limits
Input / Output	Console-based menu and formatted text output
Data Storage	In-memory STL vectors; no external database is required for this academic prototype
Platform	Windows / Linux / macOS with a C++17 compiler

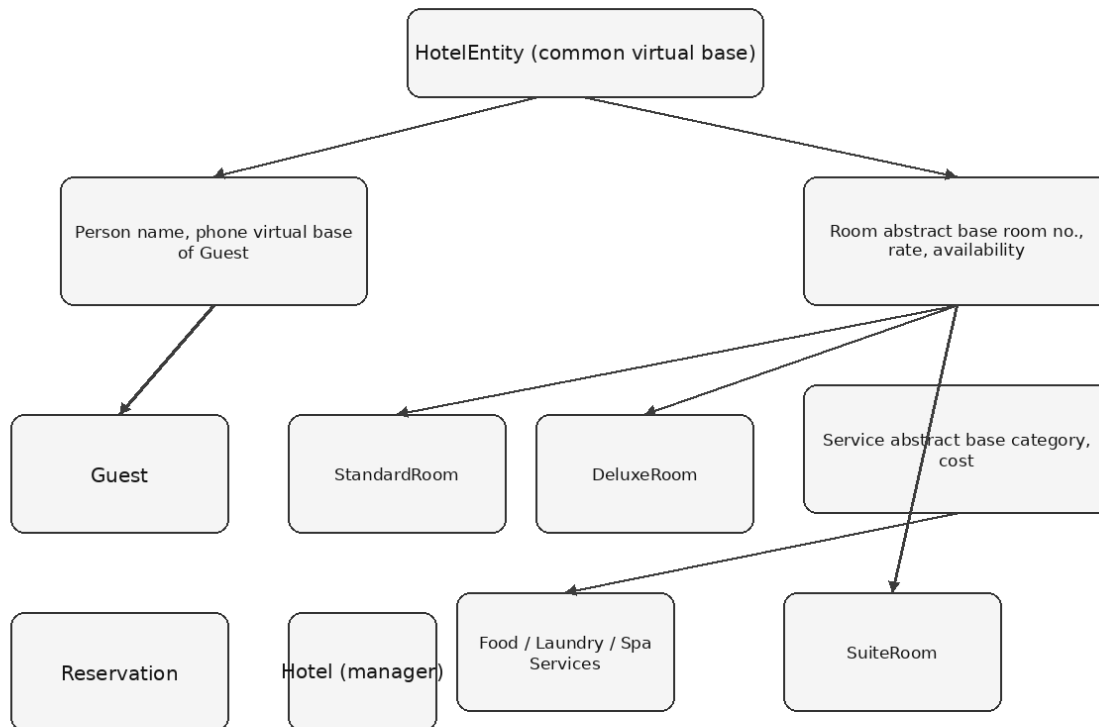
4. Design / Proposed Solution

The proposed system is organized into independent classes so that each responsibility is clearly separated. Hotel acts as the main controller and stores guests, rooms, reservations and the service catalog.

4.1 Major Classes

Class	Responsibility
HotelEntity	Common base class containing a shared ID. It is used as a virtual base in classes where multiple inheritance may occur.
Person	Stores common guest/person information such as name and phone number.
Guest	Derived from Person; stores guest email and provides guest display functionality.
Room	Abstract base class containing room number, rate and availability. It declares the pure virtual function type().
StandardRoom	Derived room type with a base nightly rate of Rs.1000.
DeluxeRoom	Derived room type with a calculated nightly rate of Rs.2000.
SuiteRoom	Derived room type with a calculated nightly rate of Rs.3500.
Service	Abstract base class for hotel services. It defines category() and cost().
FoodService	Service category for food / breakfast.
LaundryService	Service category for laundry.
SpaService	Service category for spa services.
RoomServiceBundle	Illustrative multiple-inheritance class derived from both Room and Service; virtual inheritance of HotelEntity prevents duplicate base subobjects.
Reservation	Stores reservation ID, guest ID, room number, number of nights, selected services and active/cancelled status.
Hotel	Main controller that manages the collections and implements menu operations.

Object-Oriented Class Design



Hotel owns rooms and reservations; reservations reference guests, rooms and services via pointers.

Figure 1. High-level object-oriented class design.

4.2 OOP Concepts Demonstrated

Encapsulation: Data members are kept inside classes and accessed through public member functions.

Abstraction: Room and Service are abstract classes with pure virtual functions.

Inheritance: StandardRoom, DeluxeRoom and SuiteRoom inherit from Room; service classes inherit from Service.

Runtime Polymorphism: Room and Service objects are accessed through base-class pointers/references, so overridden functions execute according to the actual object type.

Virtual Inheritance: HotelEntity is used as a virtual base in Person, Room and Service. RoomServiceBundle demonstrates how virtual inheritance avoids duplicate HotelEntity subobjects in a diamond/multiple-inheritance structure.

Dynamic Object Management: `std::unique_ptr` and `std::shared_ptr` are used instead of raw owning pointers, reducing memory-management errors.

5. Algorithm / Pseudocode / Flowchart

5.1 Main Algorithm

1. Start the application.

2. Create a Hotel object.
3. Initialize room objects and the service catalog.
4. Display the main menu.
5. Read the user's menu choice.
6. Perform the selected operation: register guest, view rooms, view services, make reservation, cancel reservation, or display reservation details/bill.
7. Validate guest ID, room availability, number of nights and service IDs where required.
8. Update room availability and reservation status.
9. Repeat the menu until the user selects Exit.
10. Terminate the program.

5.2 Reservation Pseudocode

```
BEGIN
  IF no guests exist
    display "Register a guest first"
    RETURN
  ENDIF

  INPUT guest ID, room number, nights
  FIND guest and room

  IF guest or room is invalid
    display error
    RETURN
  ENDIF

  IF room is occupied OR nights <= 0
    display error
    RETURN
  ENDIF

  CREATE reservation with new reservation ID
  ASK whether services are required

  WHILE user wants another service
    DISPLAY service catalog
    INPUT service ID
    ADD selected service to reservation
  ENDWHILE

  MARK room as occupied
  STORE reservation
  DISPLAY reservation ID
END
```

5.3 Billing Algorithm

Hotel Reservation Management Flowchart

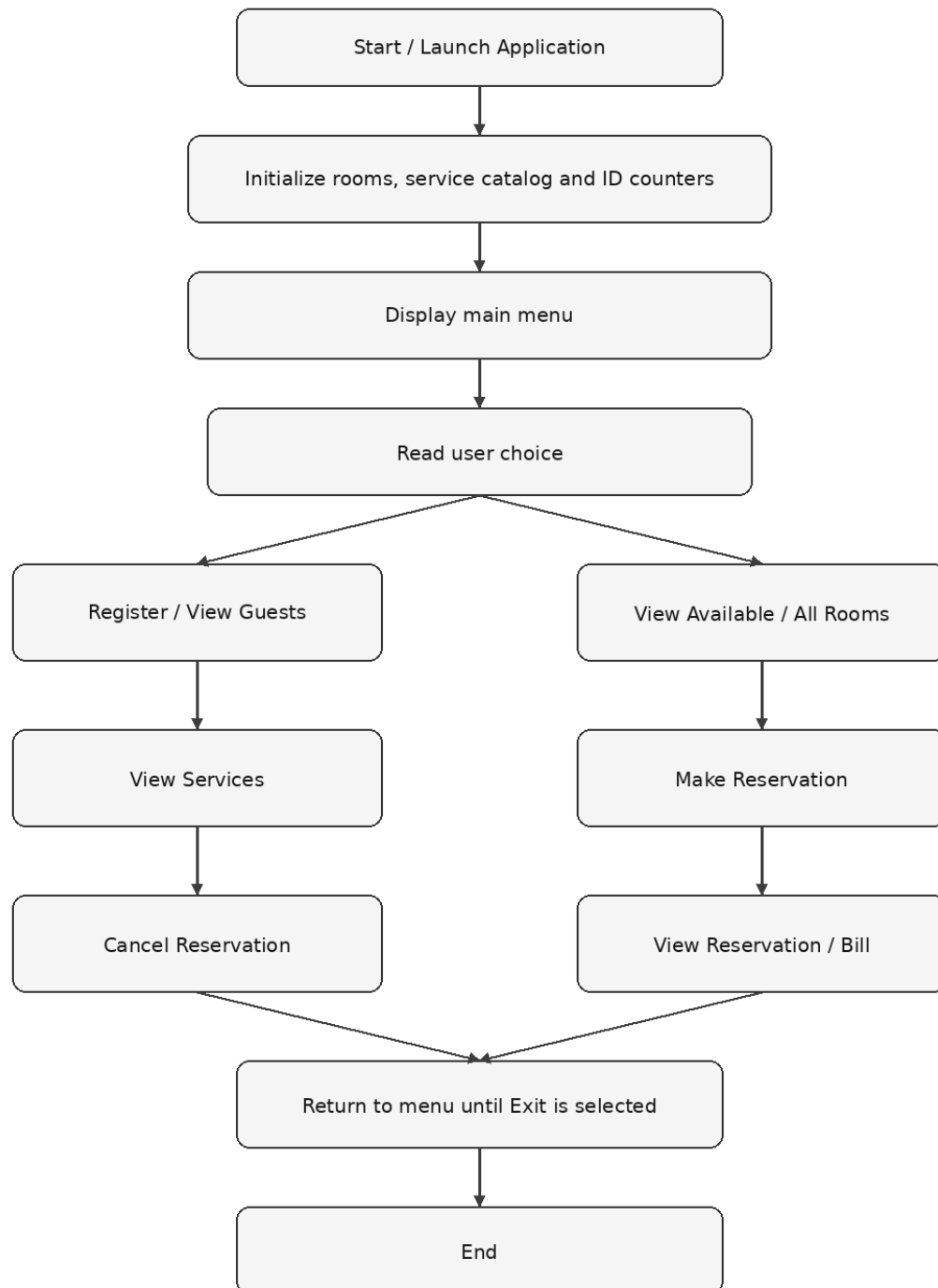


Figure 2. System flowchart.

6. Implementation

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Hotel {
```

```
private:
```

```
    int roomNo;
```

```
    string guestName;
```

```
    string phone;
```

```
    string roomType;
```

```
    int days;
```

```
    bool booked;
```

```
public:
```

```
    Hotel() {
```

```
        roomNo = 0;
```

```
        guestName = "";
```

```
        phone = "";
```

```
        roomType = "";
```

```
        days = 0;
```

```
    booked = false;

}

void setRoom(int no, string type) {

    roomNo = no;

    roomType = type;

}

void bookRoom() {

    if (booked) {

        cout << "Room is already booked!\n";

        return;

    }

    cout << "Enter Guest Name: ";

    cin >> guestName;

    cout << "Enter Phone Number: ";

    cin >> phone;

    cout << "Enter Number of Days: ";

    cin >> days;

    booked = true;

    cout << "\nRoom booked successfully!\n";
```

```

}

void cancelRoom() {

    if (!booked) {

        cout << "Room is not booked!\n";

    } else {

        booked = false;

        guestName = "";

        phone = "";

        days = 0;

        cout << "Reservation cancelled successfully!\n";

    }

}

void checkIn() {

    if (booked)

        cout << "Guest " << guestName << " checked in successfully.\n";

    else

        cout << "Room is not reserved.\n";

}

void checkOut() {

```

```

if (!booked) {

    cout << "Room is not occupied.\n";

    return;

}

int rate;

if (roomType == "Single")

    rate = 1500;

else if (roomType == "Double")

    rate = 2500;

else

    rate = 4000;

int bill = rate * days;

cout << "\n----- HOTEL BILL ----- \n";

cout << "Guest Name : " << guestName << endl;

cout << "Room No   : " << roomNo << endl;

cout << "Room Type : " << roomType << endl;

cout << "Days       : " << days << endl;

cout << "Total Bill : Rs." << bill << endl;

cout << "----- \n";

```

```

        booked = false;

        guestName = "";

        phone = "";

        days = 0;

        cout << "Check-out completed successfully!\n";

    }

    void display() {

        cout << roomNo << "\t"

            << roomType << "\t\t"

            << (booked ? "Booked" : "Available") << endl;

    }

};

int main() {

    Hotel rooms[5];

    rooms[0].setRoom(101, "Single");

    rooms[1].setRoom(102, "Single");

    rooms[2].setRoom(201, "Double");

    rooms[3].setRoom(202, "Double");

    rooms[4].setRoom(301, "Suite");

```

```

int choice, room;

do {

    cout << "\n===== \n";

    cout << " HOTEL RESERVATION MANAGEMENT SYSTEM\n";

    cout << "===== \n";

    cout << "1. View Room Availability\n";

    cout << "2. Book Room\n";

    cout << "3. Cancel Reservation\n";

    cout << "4. Check-In\n";

    cout << "5. Check-Out and Generate Bill\n";

    cout << "6. Exit\n";

    cout << "Enter your choice: ";

    cin >> choice;

    switch (choice) {

    case 1:

        cout << "\nRoom No\tRoom Type\tStatus\n";

        for (int i = 0; i < 5; i++)

            rooms[i].display();

```

```
        break;

case 2:

    cout << "Enter room number: ";

    cin >> room;

    if (room == 101)

        rooms[0].bookRoom();

    else if (room == 102)

        rooms[1].bookRoom();

    else if (room == 201)

        rooms[2].bookRoom();

    else if (room == 202)

        rooms[3].bookRoom();

    else if (room == 301)

        rooms[4].bookRoom();

    else

        cout << "Invalid room number!\n";

    break;

case 3:

    cout << "Enter room number: ";
```

```
cin >> room;

if (room == 101)

    rooms[0].cancelRoom();

else if (room == 102)

    rooms[1].cancelRoom();

else if (room == 201)

    rooms[2].cancelRoom();

else if (room == 202)

    rooms[3].cancelRoom();

else if (room == 301)

    rooms[4].cancelRoom();

else

    cout << "Invalid room number!\n";

break;
```

case 4:

```
cout << "Enter room number: ";

cin >> room;

if (room == 101)
```

```
rooms[0].checkIn();

else if (room == 102)

rooms[1].checkIn();

else if (room == 201)

rooms[2].checkIn();

else if (room == 202)

rooms[3].checkIn();

else if (room == 301)

rooms[4].checkIn();

else

cout << "Invalid room number!\n";

break;

case 5:

cout << "Enter room number: ";

cin >> room;

if (room == 101)

rooms[0].checkOut();

else if (room == 102)
```

```
rooms[1].checkOut();

else if (room == 201)

rooms[2].checkOut();

else if (room == 202)

rooms[3].checkOut();

else if (room == 301)

rooms[4].checkOut();

else

cout << "Invalid room number!\n";

break;

case 6:

cout << "Thank you for using the system!\n";

break;

default:

cout << "Invalid choice!\n";

}

} while (choice != 6);

return 0;

}
```

7. Test Cases and Expected / Actual Results

Test ID	Operation	Input	Expected Result	Actual / Status
TC01	Register guest	Enter Alice, phone and email	Guest ID 1001 generated	Pass
TC02	View all rooms	Select option 4	Five rooms displayed; all initially available	Pass
TC03	Make reservation	Guest 1001, room 201, 2 nights, Breakfast	Reservation 5001 created; room 201 becomes occupied	Pass
TC04	Calculate bill	Reservation 5001	Rs.4350.00 displayed	Pass
TC05	Cancel reservation	Reservation ID 5001	Reservation cancelled; room becomes available	Pass
TC06	Invalid room/guest	Enter an unknown guest or room	Error message displayed; no reservation created	Not executed in sample; logic implemented
TC07	Invalid nights	Enter 0 or negative nights	Validation message displayed	Not executed in sample; logic implemented
TC08	Unavailable room	Reserve an occupied room	'Room is not available' displayed	Not executed in sample; logic implemented

8. Execution Screenshots / Output

```
=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 1

Room No Room Type      Status
101      Single        Available
102      Single        Available
201      Double         Available
202      Double         Available
301      Suite          Available

=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 2
Enter room number: 201
Enter Guest Name: CHAITANYA
Enter Phone Number: 1234567890
Enter Number of Days: 2

Room booked successfully!
```

```
=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 4
Enter room number: 201
Guest CHAITANYA checked in successfully.

=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 5
Enter room number: 201

----- HOTEL BILL -----
Guest Name : CHAITANYA
Room No    : 201
Room Type  : Double
Days       : 2
Total Bill : Rs.5000
-----
Check-out completed successfully!
```

```
=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 4
Enter room number: 201
Room is not reserved.

=====
HOTEL RESERVATION MANAGEMENT SYSTEM
=====
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 6
Thank you for using the system!

-----
Process exited after 374.2 seconds with return value 0
Press any key to continue . . . |
```

9. Analysis and Discussion

9.1 Functional Analysis

The application satisfies the main functional requirements of a small hotel reservation prototype. Guests can be registered and identified by unique IDs. Room availability is tracked using a Boolean state, and a reservation changes that state from available to occupied. Cancellation reverses the state. Services are selected dynamically from a polymorphic service catalog and are included in the bill.

9.2 Object-Oriented Analysis

The abstract Room and Service classes provide common interfaces while derived classes implement category-specific behavior. Virtual functions make it possible to call `type()`, `rate()`, `category()` and `cost()` through base-class references/pointers. Virtual inheritance of `HotelEntity` prevents repeated copies of the common ID when `RoomServiceBundle` combines Room and Service. This demonstrates a practical use of virtual base classes in C++.

9.3 Limitations and Future Enhancements

- Data is stored only in memory, so records are lost when the program exits.
- A database such as MySQL or SQLite could be added for persistent storage.
- Date-based check-in/check-out and overlapping reservations could be implemented.
- User authentication and role-based access could be introduced.
- Taxes, discounts, deposits and detailed invoices could be added.
- A graphical or web interface could replace the console menu.
- Search and reporting features could be added for guests, rooms and revenue.

10. Conclusion

The Hotel Accommodation and Reservation Management System demonstrates how C++ object-oriented programming can be used to model a realistic management application. The design separates guests, rooms, services and reservations into appropriate classes and uses inheritance and runtime polymorphism to support different categories. The program successfully performs guest registration, room availability checking, reservation creation, service selection, bill calculation and reservation cancellation. The modular structure also provides a foundation for future features such as database storage, online booking, authentication and graphical interfaces.

11. References

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley.
2. Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, *C++ Primer*, 5th Edition, Addison-Wesley.
3. Herbert Schildt, *C++: The Complete Reference*, McGraw-Hill.
4. ISO/IEC C++ language concepts: classes, inheritance, virtual functions and standard library containers.
5. cppreference.com — reference material for C++ language features and the standard library.



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Team: 4

Slot: C

Course Code: DSA0112

Course Name: OBJECT ORIENTED PROGRAMMING WITH C++

Course Faculty: Dr Sudha I

Project Title: Hotel Accommodation and Reservation Management Using C++

Module Photographs: (3 photographs –Module Photo, Individual student contribution module work in the project and presentation image)



```
HOTEL RESERVATION MANAGEMENT SYSTEM
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 4
Enter room number: 201
Guest CHAITANYA checked in successfully.

HOTEL RESERVATION MANAGEMENT SYSTEM
1. View Room Availability
2. Book Room
3. Cancel Reservation
4. Check-In
5. Check-Out and Generate Bill
6. Exit
Enter your choice: 5
Enter room number: 201

----- HOTEL BILL -----
Guest Name : CHAITANYA
Room No : 201
Room Type : Double
Days : 2
Total Bill : Rs.5000
Check-out completed successfully!
```

Project Description: (here you write what you did in this project (contribution) including Model Description)

1. The hotel accommodation system continuously manages the available rooms using the program.
2. In our project, rooms are categorized based on room type, price, and availability.
3. When a customer wants to book a room, the system checks whether the selected room is available.
4. If the room is available, the system allows the customer to make a reservation.
5. If the room is already booked, the system displays a room unavailable message.
6. The system stores important reservation details such as customer name, room number, room type, and number of days.
7. The total accommodation cost is calculated based on the room price and number of days.
8. This feature helps users easily book, cancel, and manage hotel reservations without manually maintaining the accommodation records.
9. The system updates the room availability automatically after a successful booking or cancellation, making hotel reservation management easier and more efficient.

Student Signature

Guide Signature